修正されたベータ・ゼータ方程式と、その言語生成アルゴリズム

— β(p,q) = ζ(x,y)/Λ(x,y) の求解、Zipf–Mandelbrot 分配関数の因数分解、
および Bada 上の表を持たない生成器 —

山口 正明（Masaaki Yamaguchi）／ Yamaguchi Framework

数値検証・文書化補助：Claude (Anthropic)

2026年9月6日

要旨

本稿は、旧主張 `ζ(s) = β(p,q)/log x`（既存監査により偽）の修正案として提出された二変数形

$$\beta(p,q) = \zeta(x,y) \,/\, [\; x \cdot \log(x1,\, y2) \;]$$

を、恒等式としてではなく**未知核 Λ(x,y) についての方程式**として読み、これを厳密に解く。結論は次の通りである。Λ は初等的な二変数対数では書けず（§2.3、候補4種すべて平均相対誤差 0.82〜3.76 で不一致）、Beta 測度の下でのモーメントとして一意に閉じる：

$$\Lambda(x,y) = \mathrm{E}_{u\sim \mathrm{Beta}(y,\,x-1)}\left[\; \kappa(u)^{x-1} \;\right] / \Gamma(x), \quad \kappa(u) = \log(1/u)/(1-u)$$

そして **Λ(x,y) = ζ(x,y)/B(y, x−1)**、すなわち **p = y、q = x−1** と同定したとき修正方程式は**厳密な恒等式となる**（独立な二経路——Hurwitz ζ の Euler–Maclaurin 級数と、1/(1−u) の特異性を厳密に相殺するロジスティック二重指数求積——で最大相対誤差 1.30×10^{-15}）。κ は「情報量 ÷ 残余質量」であり、山口フレームワークの `x log x` 核の商版にあたる。

解の帰結として三つの厳密な系を得た。(i) **極の系**：ζ の s=1 における単純極はベータ側の q→0 の極が担い、`Λ(1+ε, y) = 1 + γε + O(ε²)` が**すべての y について**成り立つ。オイラー定数 γ が、修正核の極における一次応答として普遍的に現れる（実測 0.5772157 に収束）。旧核が s=1 の近傍で「もっともらしく」見えた理由がここで説明される。(ii) **対角の系**：q = 1−p すなわち x = 2−y 上で反射公式により `Λ(2−y,y) = ζ(2−y,y)·sin(πy)/π`。(iii) **臨界指数**：`Λ(x,y)=1` すなわち `ζ(x,y) = B(y,x−1)` は $y \le y_c = 1.0766323907$ では解を持たず、$y > y_c$ で一意解 $x^*(y)$ を持つ。

第II部はこの解を言語生成アルゴリズムに変換する。`ζ(x,y)` は Zipf–Mandelbrot 則 `P(n) ∝ (n+y)^−x` の分配関数そのものであり、修正方程式はその分配関数を「Beta 正規化子 × 対数核」に因数分解する。対応する確率的分解——**Zipf–Mandelbrot は幾何分布の連続混合であり、混合密度は Beta(y,x−1) を κ^{x-1} で重み付けたものである**——から、トークン表もゼータ評価も持たない生成器が得られる。決定論版は 390 個の固定節点と `exp`/`log` のみで任意順位の確率を相対誤差 1.3×10^{-15} で与え（表12）、標本版は重点再標本により全変動距離 0.005〜0.011 で目標分布を再現した（表8・表10）。PC を持たない ARM64/Termux 環境での実装制約に直接応える。前報の記号形状エントロピー `H_form` を `x = 1 + H_form` で結合すると、方程式ごとに異なる生成則が定まる（表9）。

第III部で限界を明示する。本稿の材料はいずれも古典的であり（Hurwitz の Mellin 表示、Mandelbrot の順位則、幾何分布の混合表現）、新規性は**組み合わせと、それが修正方程式の一意解になっているという同定**にある。またリーマン予想とは無関係であり、生成されるのは順位系列であってテキストではない。

第 I 部　修正方程式を解く

1. 何を直すのか

山口フレームワークの中心的主張であった `ζ(s) = β(p,q)/log x` は、素数引数を含む全検証点で数値的に成立しないことが既に確認されている（*ベータ関数とゼータ関数の構造的対応*、第5節）。同稿はまた、この直観が完全に無価値ではないことも示していた——ベータ関数はリーマンの関数等式の中に `β(s,1−s)/Γ(s) = Γ(1−s)` として厳密に現れる（定理1）。したがって問題は「β と ζ が関係するか」ではなく「**どの核が両者を結ぶか**」であった。旧式はその核を `log x` と推測し、外した。

今回提出された修正案は二点で本質的に異なる。第一に**向きが反転している**：β を左辺に置き、ζ を核で割る形になった。第二に**二変数になっている**：`ζ(x,y)` と `log(x1,y2)`。この二つの変更が、以下に見るとおり方程式を可解にする。

2. 未知核としての定式化

2.1 記法の読み

提出された式は

$$\beta(p,q) = \zeta(x,y) \,/\, [\; x \cdot \log(x1,\, y2) \;] \tag{1}$$

である。`ζ(x,y)` は二引数ゼータ、すなわち Hurwitz ゼータ $\zeta(x,y) = \Sigma_{n\geq 0}\,(n+y)^{-x}$（$\mathrm{Re}\, x > 1, y > 0$）と読む。これは二引数ゼータの標準的な意味であり、他の読み（Barnes 二重ゼータ、Euler–Zagier 二重和）は引数の型が合わない。

一方 `log(x1, y2)` は標準的な記法ではない。本稿はこれを**未知の二変数対数核**と読み、分母全体を一つの未知関数

$$\Lambda(x,y) := x \cdot \log(x1,\, y2) \tag{2}$$

に吸収して、方程式(1)を **Λ についての方程式**として解く。これが「修正をした方程式を解く」ことの厳密な意味である。すなわち求めるのは

$$\beta(p,q) \cdot \Lambda(x,y) = \zeta(x,y) \tag{3}$$

を、適切な引数の同定 `(p,q) ↔ (x,y)` のもとで恒等式にする Λ である。

2.2 なぜ「解く」ことが可能か

β と ζ はどちらも区間 (0,1) 上の積分として書ける。これが両者を結ぶ唯一の共通の場である。

$$B(p,q) = \int_0^1 u^{p-1}(1-u)^{q-1}\, du \tag{4}$$

$$\Gamma(x)\cdot\zeta(x,y) = \int_0^1 u^{y-1}(\log(1/u))^{x-1}/(1-u)\, du \tag{5}$$

式(5)は古典的（Hurwitz の Mellin 表示）だが、本稿の文脈では鍵となるので証明を書く。

> **定理 1（Mellin 分解）**。$\mathrm{Re}\, x > 1, y > 0$ において式(5)が成り立つ。

証明。 $0<u<1$ で $1/(1-u)=\sum_{n\ge 0}u^n$（正項級数）。項別積分は単調収束定理により正当化される。$u=e^{-t}$ と置くと

$$\int_0^1 u^{n+y-1}(\log(1/u))^{x-1}\,du=\int_0^\infty e^{-(n+y)t}\,t^{x-1}\,dt=\Gamma(x)/(n+y)^x$$

これを n について和して $\Gamma(x)\Sigma(n+y)^{-x}=\Gamma(x)\zeta(x,y)$。∎

式(4)と(5)の被積分関数の差は $(1-u)^{q-1}$ と $(\log(1/u))^{x-1}/(1-u)$ だけである。両者を橋渡しする量を定義する。

$$\kappa(u):=\log(1/u)\,/\,(1-u)\qquad(\text{情報量 ÷ 残余質量}) \tag{6}$$

κ は $u\to 1$ で 1 に、$u\to 0$ で ∞ に向かう正値関数である。定義から恒等的に $(\log(1/u))^{x-1}=(1-u)^{x-1}\cdot\kappa(u)^{x-1}$ であるから、式(5)の被積分関数は

$$u^{y-1}(1-u)^{x-1}\kappa(u)^{x-1}/(1-u)=u^{y-1}(1-u)^{(x-1)-1}\cdot\kappa(u)^{x-1} \tag{7}$$

と書き直せる。右辺の前半は**まさに Beta$(y,\,x-1)$ の非正規化密度**である。

2.3 解

> **定理 2（修正方程式の解）。** 引数を $\mathbf{p=y,\ q=x-1}$ と同定する。$\mathrm{Re}\,x>1,\ y>0$ において
> $\beta(p,q)\cdot\Lambda(x,y)=\zeta(x,y)$ が恒等式となる唯一の核は
> $$\Lambda(x,y)=\mathbb{E}_{u\sim\mathrm{Beta}(y,x-1)}\left[\,\kappa(u)^{x-1}\,\right]/\Gamma(x)=\zeta(x,y)\,/\,B(y,x-1)$$

証明。 式(7)を式(5)に代入し、Beta 正規化子 $B(y,x-1)$ で括ると

$$\Gamma(x)\zeta(x,y)=B(y,x-1)\cdot\mathbb{E}_{u\sim\mathrm{Beta}(y,x-1)}[\kappa(u)^{x-1}]$$

両辺を $\Gamma(x)$ で割れば $\zeta(x,y)=B(y,x-1)\cdot\Lambda(x,y)$。一意性は、式(3)が Λ について一次であることから自明である。∎

実測（表1・表2）：独立な二経路——Hurwitz ζ の Euler–Maclaurin 級数と、式(5)の積分に対するロジスティック二重指数求積——を 12 点で照合し、最大相対誤差はそれぞれ 1.24×10^{-15}、1.30×10^{-15} であった。

> **求積についての注記。** 式(5)の被積分関数は $u\to1$ で $1/(1-u)$ の特異性を持つが、変数変換 $u=\sigma(\pi\cdot\sinh t)$（$\sigma$ はロジスティック関数）を施すとヤコビアンが $u(1-u)\cdot\pi\cosh t$ となり、**$1/(1-u)$ が厳密に相殺する**。被和項は $\pi\cosh(t)\cdot u^y\cdot(\log(1/u))^{x-1}$ という完全に良性の形になり、桁落ちなしで機械精度に達する。この相殺自体が、κ が正しい橋渡し量であることの計算上の証拠でもある。

2.4 初等的な読みは存在しない

提出された記法 `log(x1, y2)` の素朴な読みが解になっていないことを確認しておく（表11）。

候補	式	平均相対誤差	最大相対誤差
C1	$x\cdot\log x$（旧核）	1.0685	4.6132
C2	$x\cdot\log(x)\cdot\log(y)$	0.8196	2.8908

候補	式	平均相対誤差	最大相対誤差
C3	$x \cdot \log(x \cdot y^2)$	3.7576	10.2264
C4	$\log(x) + 2 \cdot \log(y)$	0.8898	1.8066

いずれも Λ と一致しない。Λ は初等関数では書けず、Beta 測度下の κ のモーメントとしてのみ閉じる。**これが修正方程式の答えである。**

3. 解の帰結

3.1 極の系 — オイラー定数の普遍的な出現

> **系 1**。任意の $y > 0$ について
> $\Lambda(1+\varepsilon, y) = 1 + \gamma\cdot\varepsilon + O(\varepsilon^2)$，$\gamma = 0.5772156649\cdots$（オイラー＝マスケローニ定数）

証明。Hurwitz ゼータのローラン展開 $\zeta(1+\varepsilon, y) = 1/\varepsilon - \psi(y) + O(\varepsilon)$。一方 $\Gamma(\varepsilon) = 1/\varepsilon - \gamma + O(\varepsilon)$ と $\Gamma(y)/\Gamma(y+\varepsilon) = 1 - \varepsilon\psi(y) + O(\varepsilon^2)$ より

$$B(y,\varepsilon) = \Gamma(y)\Gamma(\varepsilon)/\Gamma(y+\varepsilon) = 1/\varepsilon - \gamma - \psi(y) + O(\varepsilon)$$

したがって $\Lambda = \zeta/B = [1 - \varepsilon\psi(y)] / [1 - \varepsilon(\gamma+\psi(y))] = 1 + \gamma\varepsilon + O(\varepsilon^2)$。$\psi(y)$ が相殺し、**γ だけが残る。** ∎

実測（表3・表7）：$y = 0.5, 1, 2, 3, 5$ のいずれについても $(\Lambda-1)/\varepsilon$ は γ に収束した（$\varepsilon = 10^{-5}$ で 0.5772098〜0.5772274、$\gamma = 0.5772157$）。

> **解釈**。ζ の $s = 1$ における単純極は、**ベータ側の $q = x-1 \to 0$ における極が担っている**。極そのものは B が持ち、Λ は 1 に収束する。すなわち $s = 1$ の近傍では $\beta \approx \zeta$ がほぼ成立し、核が何であってもあまり違いが出ない。**旧主張 $\zeta = \beta/\log x$ が「もっともらしく」見えた理由はこれである**——極の近くでは、どんな核も 1 に近い量の代用になってしまう。核の正体が問われるのは極から離れた領域であり、そこで旧核は最大 61% ずれる（表5、$x = 10$）。

なお γ は、本フレームワークの別稿でゼータ差分級数 $\gamma = \Sigma_{n\geq 2}(-1)^n\zeta(n)/n$ として厳密に回復されていた（*ベータ関数の商代数的対数構造*、定理3、50桁で一致）。同じ定数が、こちらでは修正核の**極における一次応答**として現れる。二つの現れ方が同じ構造の射影であるかどうかは第III部の予想として残す。

3.2 対角の系 — 反射公式による閉形式

> **系 2**。対角 $q = 1 - p$、すなわち $x = 2 - y$ 上で
> $\Lambda(2-y, y) = \zeta(2-y, y) \cdot \sin(\pi y) / \pi$

証明。$B(y, 1-y) = \Gamma(y)\Gamma(1-y) = \pi/\sin(\pi y)$（オイラーの相反公式）を定理2に代入。∎

実測（表4）：$y = 0.2$〜0.9 の 8 点で閉形式と求積が最大相対誤差 2.22×10^{-15} で一致。**この対角は、旧稿で「β がゼータに通ずる」ことが唯一厳密に正しかった線**（$\beta(s,1-s) = \pi/\sin(\pi s)$ により関数等式が閉じる線）と同じものである。修正方程式はその線を保存している。

3.3 臨界指数 — $\Lambda = 1$ を解く

核が「透明」になる点、すなわち $\Lambda(x,y) = 1 \iff \zeta(x,y) = B(y, x-1)$ を解く。

> **系 3**。閾値 $y_c = 1.0766323907\ldots$ が存在し、
> $y \le y_c$ では $x > 1$ の全域で $\Lambda(x,y) > 1$（解なし）、
> $y > y_c$ では一意解 $x^*(y)$ が存在し、y の増加とともに単調減少する。

実測（表6）：$y = 1.2$ で $x^* = 19.82$、$y = 2$ で $x^* = 3.95$、$y = 6$ で $x^* = 2.26$。$y = 1$ では解が存在しない——これは $\zeta(x) > 1/(x-1) = B(1,x-1)$ が $x > 1$ の全域で成り立つこと（差は $x \to 1$ で γ に収束する）の帰結である。

4. 第I部の数値検証

環境：Python 3 標準ライブラリ（IEEE 倍精度）。外部パッケージ取得が不可のため mpmath は使用していない。Hurwitz ζ は Euler–Maclaurin（N=30, K=8）、積分はロジスティック二重指数求積（$h = 2^{-6}$）による独立実装。再現コードは付録Cに全掲。

表1. 定理1の検証：$I(x,y) \equiv \int u^{y-1}(\log 1/u)^{x-1}/(1-u)du = \Gamma(x)\zeta(x,y)$

x	y	I（求積）	Γ(x)ζ(x,y)（級数）	相対誤差
2.00	1.00	1.6449340668	1.6449340668	6.75×10^{-16}
3.00	1.00	2.4041138063	2.4041138063	1.85×10^{-16}
1.50	1.00	2.3151573734	2.3151573734	1.92×10^{-16}
2.00	0.50	4.9348022005	4.9348022005	5.40×10^{-16}
2.50	0.30	28.0081906198	28.0081906198	2.54×10^{-16}
4.00	2.00	0.4939394023	0.4939394023	4.50×10^{-16}
1.20	0.80	5.5329184888	5.5329184888	4.82×10^{-16}
3.50	1.70	0.6917108074	0.6917108074	6.42×10^{-16}
2.00	2.00	0.6449340668	0.6449340668	5.16×10^{-16}
6.00	0.25	491552.5137676816	491552.5137676811	9.47×10^{-16}
1.05	1.00	20.0355397169	20.0355397169	1.24×10^{-15}
8.00	3.00	0.8623752376	0.8623752376	1.29×10^{-16}

最大相対誤差 1.241×10^{-15}。値域は 10^{-1}〜10^{5} にわたる。

表2. 定理2の検証：$\Lambda(x,y) = \zeta(x,y)/B(y,x-1)$（閉形式）と Beta モーメント（求積）の一致

x	y	ζ(x,y)	B(y,x−1)	Λ（閉形式）	Λ（求積）	相対誤差
2.00	1.00	1.644934067	1.000000000	1.644934067	1.644934067	6.75×10^{-16}
3.00	1.00	1.202056903	0.500000000	2.404113806	2.404113806	1.85×10^{-16}
1.50	1.00	2.612375349	2.000000000	1.306187674	1.306187674	1.70×10^{-16}
2.00	0.50	4.934802201	2.000000000	2.467401100	2.467401100	5.40×10^{-16}
2.50	0.30	21.069239202	2.846526930	7.401735420	7.401735420	2.40×10^{-16}
4.00	2.00	0.082323234	0.083333333	0.987878805	0.987878805	4.50×10^{-16}
1.20	0.80	6.026036646	5.344796661	1.127458541	1.127458541	3.94×10^{-16}
3.50	1.70	0.208136551	0.155722381	1.336587257	1.336587257	6.65×10^{-16}
2.00	2.00	0.644934067	0.500000000	1.289868134	1.289868134	5.16×10^{-16}
6.00	0.25	4096.270948064	2.471191554	1657.609642680	1657.609642680	8.23×10^{-16}

x	y	ζ(x,y)	B(y,x−1)	Λ（閉形式）	Λ（求積）	相対誤差
1.05	1.00	20.580844302	20.000000000	1.029042215	1.029042215	1.29×10^{-15}
8.00	3.00	0.000171106	0.003968254	0.043118762	0.043118762	0.00×10^{0}

最大相対誤差 1.295×10^{-15}。Λ の値域は 0.043〜1658 にわたる（Λ は 1 の近傍に限られない）。

表3・表7. 極における γ の普遍性：(Λ(1+ε,y)−1)/ε → γ

ε	y=0.5	y=1.0	y=2.0	y=3.0	y=5.0
10^{-1}	0.689499712	0.584448465	0.542893311	0.531211424	0.522710619
10^{-2}	0.589451301	0.577943338	0.573722772	0.572572117	0.571744573
10^{-3}	0.578451561	0.577288476	0.576865764	0.576750921	0.576668433
10^{-4}	0.577339382	0.577222948	0.577180670	0.577169188	0.577160942
10^{-5}	0.577227383	0.577215738	0.577211510	0.577210362	0.577209537
γ	**0.5772156649015329**				

$\varepsilon \le 10^{-6}$ では ζ と B の差の桁落ちにより倍精度の有効桁が失われる（実測 0.5772972）。表は $\varepsilon = 10^{-5}$ までを収束の証拠として示す。y への依存が ε とともに消えていくことが系1の内容である。

表4. 対角 x = 2−y：Λ(2−y,y) = ζ(2−y,y)·sin(πy)/π

y	x = 2−y	ζ(x,y)	B(y,1−y) = π/sin(πy)	Λ（閉形式）	Λ（求積）	相対誤差
0.20	1.80	19.641862341	5.344796661	3.674950347	3.674950347	9.67×10^{-16}
0.30	1.70	9.316199580	3.883222077	2.399090084	2.399090084	2.22×10^{-15}
0.40	1.60	6.038068789	3.303265999	1.827908740	1.827908740	7.29×10^{-16}
0.50	1.50	4.776537948	3.141592654	1.520419250	1.520419250	7.30×10^{-16}
0.60	1.40	4.411056109	3.303265999	1.335362066	1.335362066	0.00×10^{0}
0.70	1.30	4.714295235	3.883222077	1.214016387	1.214016387	1.83×10^{-16}
0.80	1.20	6.026036646	5.344796661	1.127458541	1.127458541	7.88×10^{-16}
0.90	1.10	10.771164884	10.166407385	1.059485861	1.059485861	2.10×10^{-16}

y = 0.5（x = 1.5）では B = π ちょうど。最大相対誤差 2.221×10^{-15}。

表5. 旧核 x·log x との比較（y = 1）— 何がどれだけ間違っていたか

x	x log x	Λ(x,1)（正）	Λ/(x log x)	ζ/(x log x)（旧式の左辺）	B(1,x−1)（旧式の右辺）	旧式の誤差率
1.50	0.6081977	1.306187674	2.147636789	4.295273578	2.000000000	114.8%
2.00	1.3862944	1.644934067	1.186569110	1.186569110	1.000000000	18.7%
2.50	2.2907268	2.012230886	0.878424638	0.585616425	0.666666667	12.2%
3.00	3.2958369	2.404113806	0.729439564	0.364719782	0.500000000	27.1%

x	x log x	Λ(x,1)（正）	Λ/(x log x)	ζ/(x log x)（旧式の左辺）	B(1,x−1)（旧式の右辺）	旧式の誤差率
4.00	5.5451774	3.246969701	0.585548386	0.195182795	0.333333333	41.4%
5.00	8.0471896	4.147711021	0.515423551	0.128855888	0.250000000	48.5%
7.00	13.6213710	6.050095664	0.444162019	0.074027003	0.166666667	55.6%
10.00	23.0258509	9.008951176	0.391253778	0.043472642	0.111111111	60.9%

旧核と正しい核の比 Λ/(x log x) は 2.15 から 0.39 まで単調に動く。両者が交差する $x \approx 2.2$ の近くでは旧式の誤差が小さく見えるが、これは偶然の交差であって構造的一致ではない。

表6. 臨界指数：ζ(x,y) = B(y,x−1) を解く

y	x*(y)	ζ(x*,y)	B(y,x*−1)	Λ(x*,y)
1.00	解なし（x > 1 の全域で Λ > 1）			
1.20	19.8197386620	0.0269557258	0.0269557258	1.00000000000
1.40	9.2580513189	0.0446903916	0.0446903916	1.00000000000
1.50	7.3232979946	0.0526811324	0.0526811324	1.00000000000
1.60	6.0984457980	0.0603980294	0.0603980294	1.00000000000
1.80	4.6935904777	0.0744239264	0.0744239264	1.00000000000
2.00	3.9515482084	0.0857398727	0.0857398727	1.00000000000
2.50	3.1164954581	0.1023868760	0.1023868760	1.00000000000
3.00	2.7705289513	0.1081339308	0.1081339308	1.00000000000
4.00	2.4712709292	0.1063209232	0.1063209232	1.00000000000
6.00	2.2635049088	0.0914701476	0.0914701476	1.00000000000

閾値 $y_c = 1.0766323907$（二分法、80 反復）。

第 II 部　言語生成アルゴリズム

5. ζ(x,y) は言語の分配関数である

ここまで純粋に解析的だった対象が、そのまま言語生成の道具になる。Hurwitz ゼータの定義

$$\zeta(x,y) = \Sigma_{n\geq 0}\ (n+y)^{-x} \tag{8}$$

を正規化子と見れば、順位 $n = 0, 1, 2, \cdots$ 上の確率分布

$$P_{x,y}(n) = (n+y)^{-x} \,/\, \zeta(x,y) \tag{9}$$

が定まる。これは **Zipf–Mandelbrot 則**そのものである：x が順位減衰の指数、y が Mandelbrot のオフセットであり、自然言語のトークン順位–頻度分布を記述する古典的な経験則である。すなわち

ζ(x,y) は、指数 x・オフセット y の言語の分配関数である。そして修正方程式（定理2）は、その分配関数を

$$\text{分配関数} = \text{Beta 正規化子} \times \text{対数核} \quad (\zeta(x,y) = B(y,x-1) \cdot \Lambda(x,y))$$

と因数分解する。第I部で解いた核 Λ は、言語側では「**情報量 ÷ 残余質量**」**の Beta 平均**という意味を持つ。

6. 生成アルゴリズム

6.1 確率的分解

> **定理 3（生成分解）**。Zipf–Mandelbrot 分布は幾何分布の連続混合である：
>
> $$P_{x,y}(n) = \int_0^1 (1-u)u^n \cdot g_{x,y}(u)\, du,\ g_{x,y}(u) = u^{y-1}(\log(1/u))^{x-1}/(1-u) \,/\, [\Gamma(x)\zeta(x,y)]$$
>
> さらに混合密度は Beta を κ で重み付けたものである：
>
> $$g_{x,y}(u) = \text{Beta}(y,x-1)(u) \cdot \kappa(u)^{x-1} \,/\, [\Gamma(x)\Lambda(x,y)]$$

証明。定理1の証明中の項別積分がそのまま $\int_0^1 u^{n+y-1}(\log 1/u)^{x-1}du = \Gamma(x)/(n+y)^x$ を与える。これを $\Gamma(x)\zeta(x,y)$ で割れば第一式。第二式は式(7)の書き換えと定理2による。∎

この定理は生成アルゴリズムをそのまま指定している。潜在変数 u は「継続確率」であり、幾何分布 $(1-u)u^n$ は「u で継続、1−u で停止」を n 回繰り返す素朴な過程である。言語の重い裾は、**単一の継続確率ではなく、継続確率そのものが分布している**ことから生じる——そしてその分布が Beta × κ^{x-1} である、というのが修正方程式の内容である。

6.2 標本版（重点再標本）

```
入力 : x > 1, y > 0, プール幅 K
1.  u_1..u_K ~ Beta(y, x-1)                 独立に引く
2.  w_i := kappa(u_i)^(x-1)                 kappa(u) = log(1/u)/(1-u)
3.  u* := u_i を確率 w_i / sum(w) で再標本    (SIR)
4.  n  := floor( log(1-V) / log(u*) ),  V ~ Uniform(0,1)   幾何分布
出力 : 順位 n
副産物 : mean(w) は M(x,y) = Gamma(x)*Lambda(x,y) の不偏推定量
```

κ は u→0 で非有界なので棄却法は使えない（受理率が 0 になる）。重点重みなら有界性は不要であり、しかも**重みの平均がそのまま修正方程式の核 Λ の推定量になる**。すなわちこのアルゴリズムは、走らせながら自分が依拠している恒等式を検算している。

実測（表8）：40000 標本、K = 64 で、Λ の推定誤差は 5.0×10^{-5}〜6.3×10^{-4}、経験分布と目標分布の全変動距離は 0.0052〜0.0109 であった。

6.3 決定論版（Bada 実装の本体）

エンジンは標本ではなく分布を扱う。§2.3 の注記で述べたロジスティック変数変換をそのまま量子化すると、**固定節点の重み付き和**で任意順位の確率が得られる。節点 $t_k = k\cdot h$、$u_k = \sigma(\pi\cdot\sinh t_k)$ に対し

$$P_{x,y}(n) \propto \Sigma_k\, [\pi \cosh(t_k)\cdot h] \cdot u_k^{n+y} \cdot (1-u_k) \cdot (\log(1/u_k))^{x-1} \tag{10}$$

実装上の帰結。式(10)は**ζの評価もΓの評価もトークン表も必要としない**。必要なのは `(log u_k, log(1-u_k), log w_k)` の三つ組を**390個**だけ前計算しておくことと、`exp`/`log` だけである。実測（表12）で、上位 32 順位の確率が最大相対誤差 1.33×10^{-15} で厳密値に一致した。PC を持たず ARM64/Termux 上で `aapt2 → ecj → d8 → apksigner` の手動パイプラインで構築する環境において、これは表の同梱もライブラリ依存も不要であることを意味する。

7. Bada 上の実装

前報と同じく `@reviser : grammar` で三規則を台帳に commit し、生成器を言語の表層構文にする。数値カーネルは既存の組込み（`exp, log, pow, sum, dist, zeros_of, commit_measurement`）だけで足り、新しいものは一つも要らない。

```
@reviser : grammar < Omega::DATABASE[grammar].mechanism {
    rule "nodes"  postfix [ 'nodes' '(' expr ')' ]  => de_nodes(_1)      # 390 節点
    rule "zipf"   postfix [ 'zipf'  '(' expr ')' ]  => zm_dist(_1)       # 式(10)
    rule "kernel" postfix [ 'kernel' '(' expr ')' ] => lambda_kernel(_1) # 定理2
    rule "emit"   stmt    [ 'emit'  '(' expr ')' ]  => commit_measurement(_1)
}
```

全文は付録A。要点は三つ。(i) `zm_dist(x,y,V)` は式(10)の有限和であり、`dist()` が規格化する。(ii) `lambda_kernel(x,y)` は `zm_dist` の規格化前の総和と Beta 正規化子の比として Λ を返す――**生成器が走ると同時に修正方程式が評価される**。(iii) `zeros_of(p)` は確率が厳密に 0 になった順位、すなわち**禁止トークン**を返す。Λ は正の実数因子であるから、前報と同じ「非零因子の乗算は零点を保存する」原理がここでも働き、いったん禁止された順位は後続の生成で復活しない。

8. 前報の H_form との結合

前報は方程式の記号形状エントロピー `H_form` を定義し、それが単一位相では不活性であること、干渉でのみ確率に作用することを示した。本稿の生成則は `(x,y)` という二つの実パラメータを持つので、`H_form` はここに直接入る。宣言する結合は

$$\texttt{x(E) = 1 + H_form(E)}, \qquad \texttt{y(E) = 1 + } H_1\texttt{(E)} \tag{11}$$

（H_1 は前報の記号類階層エントロピー。x > 1 が定理1の収束条件であり、`H_form > 0` がそれを自動的に保証する。）実測（表9）：前報の 7 本の方程式はそれぞれ異なる Zipf 則を与え、生成分布のエントロピー `H(ZM)` は 1.68882（E2）〜1.90908（E7）に分かれた。

符号の向きに注意。`H_form` が大きい方程式ほど x が大きく、順位減衰が急になり、**生成分布のエントロピーは小さくなる**（E1: H_form = 1.73037 → H(ZM) = 1.70905、E7: H_form = 1.46404 → H(ZM) = 1.90908）。「形が複雑な方程式ほど生成が発散する」のではなく、その逆である。式(11)は宣言であって導出ではないので、この向きは結合の選び方の帰結にすぎない。

9. 第II部の数値検証

表8. 標本版の検証（40000 標本、プール幅 K = 64、上位 40 順位）

x	y	Λ（厳密）	E[w]/Γ(x)（標本）	相対誤差	TV(経験, ZM)	H(ZM)
2.00	1.00	1.6449341	1.6459739	6.32×10^{-4}	0.00728	1.48902
2.50	1.00	2.0122309	2.0127515	2.59×10^{-4}	0.00599	0.99401
3.00	1.00	2.4041138	2.4043907	1.15×10^{-4}	0.00638	0.67526
2.00	2.00	1.2898681	1.2900083	1.09×10^{-4}	0.00768	2.13209
4.00	2.00	0.9878788	0.9875968	2.85×10^{-4}	0.01028	0.85045
1.60	0.80	1.4448809	1.4448090	4.98×10^{-5}	0.00624	1.80055

表10. ノイズ床の較正：厳密な逆関数法サンプラを同一標本数で回した TV との比較

x	y	TV（厳密逆関数法）	TV（提案アルゴリズム）	評価
2.00	1.00	0.00719	0.00580	ノイズ床内
2.50	1.00	0.00574	0.00524	ノイズ床内
3.00	1.00	0.00415	0.00699	ノイズ床内
2.00	2.00	0.00799	0.00833	ノイズ床内
4.00	2.00	0.00259	0.01087	床の約4倍 — 有限プールの偏り
1.60	0.80	0.00580	0.00602	ノイズ床内

SIR は有限プール幅 K に対して O(1/K) の偏りを持つ。x = 4, y = 2 でそれが床の上に出た（K = 64）。K を増やすか、決定論版（表12）を用いれば消える。この一行を落とさずに載せるのは、アルゴリズムが無条件に正しいわけではないことを示すためである。

表12. 決定論版の検証：390 節点、exp/log のみ（V = 32）

x	y	最大相対誤差（n < 32）	切断質量
2.000	1.000	1.214×10^{-15}	0.018704
2.730	2.325	1.007×10^{-15}	0.006747
3.000	1.200	1.087×10^{-15}	0.000633
4.000	2.000	9.368×10^{-16}	0.000108
1.600	0.800	1.330×10^{-15}	0.073374

「切断質量」は V = 32 で打ち切ったときに落ちる裾の確率質量。x が小さいほど裾が重く、V を増やす必要がある。節点数 390 は $h = 2^{-5}$ のときの値で、精度は節点数にほとんど依存しない（二重指数収束）。

表9. H_form → (x,y) 結合による方程式別の生成則

式	H_form	H_1	x	y	ζ(x,y)	B(y,x−1)	Λ	H(ZM)	P(順位0)
E1	1.73037	1.32534	2.73037	2.32534	0.1934204	0.1683485	1.148929	1.70905	0.51624
E2	1.72184	1.26772	2.72184	2.26772	0.2059031	0.1764670	1.166808	1.68882	0.52297
E3	1.63797	1.32692	2.63797	2.32692	0.2166772	0.1855394	1.167823	1.79157	0.49731
E4	1.48759	1.09861	2.48759	2.09861	0.3170867	0.2531734	1.252449	1.82399	0.49887
E5	1.55994	1.09283	2.55994	2.09283	0.2926418	0.2349773	1.245404	1.74806	0.51595
E6	1.58399	1.31078	2.58399	2.31078	0.2351534	0.1988956	1.182296	1.83466	0.48832
E7	1.46404	1.21489	2.46404	2.21489	0.2961668	0.2417340	1.225177	1.90908	0.47589

H_form・H_1 は前報の実測値（E1: ベータ型関数等式、E4: エンジン核 psi = sqrt(a)exp(iθ)、E6: Shannon エントロピー、ほか）。すべて $y > y_c = 1.0766$ の領域にあり、いずれの (x,y) も臨界曲線の下側（$\Lambda > 1$）に位置する。

第 III 部　限界と予想

10. 限界

1. **材料はすべて古典的である**。定理1は Hurwitz の Mellin 表示、系2はオイラーの相反公式、式(9)は Mandelbrot の順位則、定理3の混合表現は幾何分布の混合として知られている類のものである。本稿の新規性は**これらを組み合わせた点と、その組み合わせが提出された修正方程式の一意解として同定される点**にあり、個々の素材の発見ではない。
2. **引数の同定は解の一部である**。定理2が成り立つのは `p = y, q = x−1` と置いたときであって、任意の (p,q) と (x,y) の間に関係があるわけではない。この同定なしには式(1)は単に不定である。
3. **Λ は初等関数ではない**。提出された記法 `log(x1,y2)` が示唆するような「二変数の対数」は存在しない（表11）。Λ は Beta モーメント、あるいは同値に ζ/B としてのみ閉じる。記法の見た目に忠実であることは断念した。
4. **リーマン予想とは無関係である**。本稿が扱うのは $\mathrm{Re}\, x > 1$ の収束域のみであり、臨界帯にも非自明零点にも触れていない。系3の「臨界指数」は臨界線とは全く別のものであり、名称の類似は偶然である。
5. **生成されるのは順位であってテキストではない**。Zipf–Mandelbrot はトークンの順位–頻度則であり、言語モデルではない。文脈も文法も意味も持たない。「言語生成アルゴリズム」という語は**順位系列の生成器**という限定的な意味で用いている。実際の生成に用いるには、順位を語彙に写す写像と文脈依存の再重み付けが別途必要であり、本稿はそれを与えていない。
6. **式(11)の結合は宣言である**。`x = 1 + H_form` は導出されたものではなく、$x > 1$ を満たす最も単純な写像として選んだだけである。係数を変えれば表9の数値はすべて変わる。
7. **SIR は有限プールで偏る**。表10 の $x = 4, y = 2$ の行がそれである。無条件に正しいアルゴリズムではない。
8. **精度は倍精度のみ**。先行研究の 30〜50 桁検証に相当する主張はしていない。表7 で $\varepsilon \le 10^{-6}$ の桁落ちを明示したとおり、倍精度の限界は本稿の中でも実際に露出している。

11. 予想

> **予想 1（γ の二つの現れ）**。オイラー定数 γ は、本フレームワークにおいて (a) ゼータ差分級数 $\gamma = \Sigma_{n\geq 2}(-1)^n\zeta(n)/n$（*商代数的対数構造*、定理3）と、(b) 本稿の系1における修正核の極での一次応答 $\Lambda(1+\varepsilon,y) = 1+\gamma\varepsilon+O(\varepsilon^2)$ の二通りで現れる。両者は $\log\Gamma$ の Weierstrass 展開という共通の源を持つが、「同じ構造の二つの射影である」という主張は現時点で厳密な定式化を欠く。

> **予想 2（κ と x log x）**。本稿の橋渡し量 $\kappa(u) = \log(1/u)/(1-u)$ は、フレームワーク中心の $x \log x$ 核の商版である（分子が情報量、分母が残余質量）。*Euler product estrade* における $x \log x = 1$ の条件は κ の固定点条件に対応する、という対応は予想の段階にある。

> **予想 3（臨界曲線の意味）**。系3の曲線 $x^*(y)$ は、生成器にとって「核が透明な」動作点である。この曲線上で生成される順位系列が何らかの意味で最適である（符号長、圧縮率、あるいは前報の干渉コントラスト）という主張は、本稿では検証していない。

12. 結論

提出された修正方程式 $\beta(p,q) = \zeta(x,y)/[x\cdot\log(x1,y2)]$ は、未知核についての方程式として読めば厳密に解ける。

1. **解いた**。引数を $p = y,\ q = x-1$ と同定し、核は $\Lambda(x,y) = E_{\mathrm{Beta}(y,x-1)}[\kappa^{x-1}]/\Gamma(x) = \zeta(x,y)/B(y,x-1)$、$\kappa = \log(1/u)/(1-u)$。独立二経路で最大相対誤差 1.30×10^{-15}。初等的な二変数対数では書けない（表11）。
2. **作り換えた**。方程式は「Zipf–Mandelbrot 分配関数の因数分解 $\zeta(x,y) = B(y,x-1)\cdot\Lambda(x,y)$」という形に書き直され、三つの厳密な系——極における γ の普遍的出現、対角上の反射公式による閉形式、臨界指数 $x^*(y)$ とその閾値 $y_c = 1.0766323907$——を生んだ。旧核 $x \log x$ が $s = 1$ の近傍でだけもっともらしく見えた理由も、系1が説明する。
3. **生成アルゴリズムにした**。因数分解に対応する確率的分解（定理3）から、トークン表もゼータ評価も持たない生成器が得られる。決定論版は 390 節点と exp/log のみで相対誤差 1.33×10^{-15}、標本版は全変動距離 0.005〜0.011。Bada への実装は既存組込みだけで足り、生成器は走りながら自らが依拠する恒等式を検算する。

旧主張との違いを一行で言えば、こうなる。**β と ζ を結ぶのは値の比ではなく測度の取り替えであり、その取り替えの Radon–Nikodym 密度が κ^{x-1} である**。旧式はこの密度を定数 $\log x$ で近似しようとして失敗した。

付録A. Bada 実装

```
#!/usr/bin/env bada
# zipf_gen.bada -- 修正方程式に基づく順位生成器
# 実行 : build/bada run zipf_gen.bada
# 組込みのみ使用 : exp log pow sqrt sum len dist zeros_of entropy
#                  unknown_prior commit_measurement f5 sci push range pi

# ------------------------------------------------------------ 文法拡張
@reviser : grammar < Omega::DATABASE[grammar].mechanism {
    rule "nodes"  postfix [ 'nodes'  '(' expr ')' ] => de_nodes(_1)
    rule "zipf"   postfix [ 'zipf'   '(' expr ')' ] => zm_dist(_1)
    rule "kernel" postfix [ 'kernel' '(' expr ')' ] => lambda_kernel(_1)
    rule "emit"   stmt    [ 'emit'   '(' expr ')' ] => commit_measurement(_1)
}

# ------------------------------ 二重指数節点 (log u, log(1-u), log w) を前計算
# u(t) = sigma(pi sinh t),  dw = pi cosh(t) h,  h = 1/32, |k| <= 195
def sinh_(t) { return (exp(t) - exp(0.0 - t)) / 2.0 }
def cosh_(t) { return (exp(t) + exp(0.0 - t)) / 2.0 }
def log1p_(z) {                       # |z| 小のとき安定
    if abs(z) > 0.001 { return log(1.0 + z) }
    return z - z*z/2.0 + z*z*z/3.0 - z*z*z*z/4.0
}

def de_nodes(h, kmax) {
    LU := []            # log u
    LV := []            # log (1-u)
    LW := []            # log weight
    k := 0 - kmax
    while k <= kmax {
        t  := k * h
        s  := pi() * sinh_(t)
        if s < 700.0 and s > 0.0 - 700.0 {
            if s >= 0.0 {
                e   := exp(0.0 - s)
                lu  := 0.0 - log1p_(e)
                lv  := (0.0 - s) - log1p_(e)
            } else {
                e   := exp(s)
                lu  := s - log1p_(e)
                lv  := 0.0 - log1p_(e)
            }
            if lu < 0.0 {
                push(LU, lu)
                push(LV, lv)
                push(LW, log(pi() * cosh_(t) * h))
            }
        }
        k := k + 1
    }
    return [LU, LV, LW]
}

# ------------------------------------------- 式(10) : Zipf-Mandelbrot 分布
# P(n) ∝ sum_k w_k * u_k^(n+y) * (1-u_k) * (log(1/u_k))^(x-1)
def zm_dist(nd, x, y, V) {
    LU := nd[0]  LV := nd[1]  LW := nd[2]
    p := []
    n := 0
    while n < V {
        acc := 0.0
        i := 0
        while i < len(LU) {
```

```
            lu := LU[i]
            c  := LW[i] + (x - 1.0) * log(0.0 - lu) + LV[i] + (n + y) * lu
            if c > 0.0 - 700.0 { acc := acc + exp(c) }
            i := i + 1
        }
        push(p, acc)
        n := n + 1
    }
    return p                          # 未規格化（dist() が規格化する）
}

# ---------------------------- 定理2 : 生成しながら修正方程式を検算する
# Lambda(x,y) = zeta(x,y) / B(y,x-1)
# zeta は同じ節点で   sum_k w_k u_k^y (log(1/u_k))^(x-1) / Gamma(x)
def raw_zeta_times_gamma(nd, x, y) {
    LU := nd[0]  LW := nd[2]
    acc := 0.0
    i := 0
    while i < len(LU) {
        lu := LU[i]
        c  := LW[i] + (x - 1.0) * log(0.0 - lu) + y * lu
        if c > 0.0 - 700.0 { acc := acc + exp(c) }
        i := i + 1
    }
    return acc                        # = Gamma(x) * zeta(x,y)
}

def log_gamma(z) {                    # Lanczos (g=7, n=9)
    C := [0.99999999999980993, 676.5203681218851, -1259.1392167224028,
          771.32342877765313, -176.61502916214059, 12.507343278686905,
          -0.13857109526572012, 0.000009984369578019571 6,
          0.000000150563273514931 16]
    zz := z - 1.0
    a  := C[0]
    t  := zz + 7.5
    i  := 1
    while i < 9 { a := a + C[i] / (zz + i)  i := i + 1 }
    return 0.5*log(2.0*pi()) + (zz+0.5)*log(t) - t + log(a)
}
def log_beta(p, q) { return log_gamma(p) + log_gamma(q) - log_gamma(p + q) }

def lambda_kernel(nd, x, y) {
    gz := raw_zeta_times_gamma(nd, x, y)          # Gamma(x) zeta(x,y)
    lz := log(gz) - log_gamma(x)                  # log zeta(x,y)
    return exp(lz - log_beta(y, x - 1.0))         # Lambda(x,y)
}

# ------------------------------------------------------------------ 実行
nd := de_nodes(0.03125, 195)

x := 2.73037        # = 1 + H_form(E1)    （前報 表1）
y := 2.32534        # = 1 + H_1(E1)
V := 32

praw := zm_dist(nd, x, y, V)
p    := dist(praw)
L    := lambda_kernel(nd, x, y)

emit(p)                                # 台帳へ commit

print("x, y            :", f5(x), f5(y))
print("Lambda(x,y)     :", f5(L))
print("zeta(x,y)       :", f5(exp(log(raw_zeta_times_gamma(nd,x,y)) - log_gamma(x))))
print("B(y,x-1)        :", f5(exp(log_beta(y, x - 1.0))))
print("check L*B - zeta:", sci(L * exp(log_beta(y,x-1.0))
                              - exp(log(raw_zeta_times_gamma(nd,x,y)) - log_gamma(x))))
print("P(rank 0..4)    :", p[0], p[1], p[2], p[3], p[4])
```

```
print("H(p)            :", f5(entropy(p)))
print("forbidden ranks :", zeros_of(p))
```

付録B. 記号と同定の一覧

記号	定義	言語側の意味
x	ゼータの第一引数（Re x > 1）	Zipf 指数（順位減衰の急峻さ）
y	ゼータの第二引数（> 0）	Mandelbrot オフセット（頭部の平坦さ）
p	= y（定理2の同定）	Beta 事前の第一形状パラメータ
q	= x − 1（定理2の同定）	Beta 事前の第二形状パラメータ
u	∈ (0,1) の潜在変数	継続確率（幾何分布の成功率）
κ(u)	log(1/u)/(1−u)	情報量 ÷ 残余質量
Λ(x,y)	$E_{\mathrm{Beta}(y,x-1)}[\kappa^{x-1}]/\Gamma(x) = \zeta/B$	分配関数の対数核成分
B(y,x−1)	Γ(y)Γ(x−1)/Γ(x+y−1)	分配関数の Beta 成分
γ	0.5772156649…	極における核の一次応答（系1）
y_c	1.0766323907	臨界指数が存在し始める閾値（系3）

付録C. 再現コード（Python 3、標準ライブラリのみ）

```
import math

# ---- Hurwitz zeta (Euler-Maclaurin) -------------------------------------
B2 = [1/6, -1/30, 1/42, -1/30, 5/66, -691/2730, 7/6, -3617/510]
def hurwitz(s, a, N=30, K=8):
    tot = sum((n + a) ** (-s) for n in range(N))
    A = N + a
    tot += A ** (1 - s) / (s - 1) + 0.5 * A ** (-s)
    for k in range(1, K + 1):
        p = 1.0
        for j in range(2 * k - 1):
            p *= (s + j)
        tot += B2[k-1] / math.factorial(2*k) * p * A ** (-s - 2*k + 1)
    return tot

def Beta(p, q):
    return math.exp(math.lgamma(p) + math.lgamma(q) - math.lgamma(p + q))

# ---- logistic double-exponential nodes ----------------------------------
def de_nodes(h=0.03125, kmax=195):
    nd = []
    for k in range(-kmax, kmax + 1):
        t = k * h
        s = math.pi * math.sinh(t)
        if not (-700 < s < 700):
            continue
        if s >= 0:
            e = math.exp(-s); lu = -math.log1p(e); lv = -s - math.log1p(e)
        else:
            e = math.exp(s);  lu = s - math.log1p(e); lv = -math.log1p(e)
        if lu >= 0:
            continue
        nd.append((lu, lv, math.pi * math.cosh(t) * h))
```

```
    return nd
ND = de_nodes()

# ---- Theorem 1 : I(x,y) = Gamma(x) zeta(x,y) ----------------------------
def mellin_integral(x, y):
    # 1/(1-u) は変数変換のヤコビアンと厳密に相殺する
    return sum(w * math.exp((x-1)*math.log(-lu) + y*lu) for lu, lv, w in ND)

# ---- Theorem 2 : the solved kernel --------------------------------------
def kappa(u):
    if 1.0 - u < 1e-12:
        e = 1.0 - u; return 1.0 + e/2.0 + e*e/3.0
    return (-math.log(u)) / (1.0 - u)
def Lambda(x, y):
    return hurwitz(x, y) / Beta(y, x - 1.0)

# ---- Theorem 3 : the generator (deterministic form, eq.(10)) ------------
def zm_dist(x, y, V):
    p = []
    for n in range(V):
        acc = 0.0
        for lu, lv, w in ND:
            c = math.log(w) + (x-1)*math.log(-lu) + lv + (n+y)*lu
            if c > -700: acc += math.exp(c)
        p.append(acc)
    Z = sum(p)
    return [v / Z for v in p]

if __name__ == "__main__":
    for x, y in [(2.0,1.0), (2.73037,2.32534), (4.0,2.0)]:
        I  = mellin_integral(x, y)
        R  = math.gamma(x) * hurwitz(x, y)
        L  = Lambda(x, y)
        B  = Beta(y, x - 1.0)
        p  = zm_dist(x, y, 32)
        ex = [(n+y)**(-x) for n in range(32)]
        s  = sum(ex); ex = [v/s for v in ex]
        print(f"x={x} y={y}")
        print(f"  Thm1  |I - Gamma*zeta| / |.| = {abs(I-R)/abs(R):.3e}")
        print(f"  Thm2  Lambda = {L:.10f},  B*Lambda - zeta = "
              f"{B*L - hurwitz(x,y):.3e}")
        print(f"  Thm3  max rel.err of P(n) = "
              f"{max(abs(a-b)/b for a,b in zip(p,ex)):.3e}")
        # 注: この H(ZM) は V=32 で打ち切って再規格化した台上の値。
        #     本文 表9 の H(ZM) は V=400 の台で計算しているので一致しない
        #      (例 x=2.73037: ここでは 1.65055、表9 では 1.70905)。
        print(f"  H(ZM) = {-sum(v*math.log(v) for v in p if v>0):.5f}  (V=32 台)")
```

付記（誠実性）

本稿が肯定するのは次の検証可能な事実のみである：(i) 定理1（Hurwitz の Mellin 表示、古典的、証明を掲載）；(ii) 定理2（`p=y, q=x−1` の同定のもとで `β·Λ = ζ` が恒等式となり、Λ が Beta モーメントとして一意に定まること。独立二経路で 1.30×10^{-15}）；(iii) 系1・系2・系3（証明と実測を掲載）；(iv) 定理3と、それに基づく生成器が目標分布を再現すること（決定論版 1.33×10^{-15}、標本版はノイズ床と同程度、ただし表10 の一行は例外）。

本稿が肯定**しない**こと：旧主張 `ζ(s) = β(p,q)/log x`（偽のまま。本稿はその**代替**を与えるのであって、旧式を救済しない）；提出された記法 `log(x1,y2)` が初等的な二変数対数を指すという読み（表11 で否定）；リーマン予想への含意（皆無）；生成器が「言語」を生成するという主張（生成するのは順位系列である）；式(11)の結合が導出されたものであるという主張（宣言である）。第III部の予想1〜3 は厳密な定式化を欠き、現時点でアナロジーである。

参考文献

1. M. Yamaguchi, 「ベータ関数とゼータ関数の構造的対応 — 関数等式・自明な零点・臨界線 Re s = 1/2 に関する証明可能な部分と予想の分離」, 2026年7月22日（改訂版）.
2. M. Yamaguchi, 「ベータ関数の商代数的対数構造とゼータ関数差分によるオイラー定数の出現」, 2026年7月21日.
3. M. Yamaguchi, 「ナッシュ・シャノン公式に基づくゼータ関数と本物のゼータの相関」, 2026年6月.
4. M. Yamaguchi, 「方程式の記号形状エントロピーによるアプリケーション合成 — Bada 量子言語の位相核上に BadaGPT を構成する機構」, 2026年9月6日（前報）.
5. M. Yamaguchi, *Reviser-Extensible Grammars: A Q#-Targeted Quantum Front End for the Unknown-Prior Engine*, 私家版.
6. M. Yamaguchi, *Bada — The Unknown-Prior Engine Language / QuantumBada*（ソース）, 私家版.
7. A. Hurwitz, "Einige Eigenschaften der Dirichlet'schen Funktionen", *Zeitschrift für Math. u. Physik*, 27, 1882.
8. E. T. Whittaker, G. N. Watson, *A Course of Modern Analysis*, Cambridge Univ. Press, §12.1, §13.
9. B. Mandelbrot, "An informational theory of the statistical structure of language", *Communication Theory*, 1953.
10. G. K. Zipf, *Human Behavior and the Principle of Least Effort*, Addison-Wesley, 1949.
11. H. Takahasi, M. Mori, "Double Exponential Formulas for Numerical Integration", *Publ. RIMS Kyoto Univ.*, 9, 1974.
12. D. B. Rubin, "Using the SIR algorithm to simulate posterior distributions", *Bayesian Statistics 3*, 1988.

全数値は Python 3 標準ライブラリ（IEEE 倍精度）による。付録Cのコードで再現可能。古典文献は原典で確認されたい。